

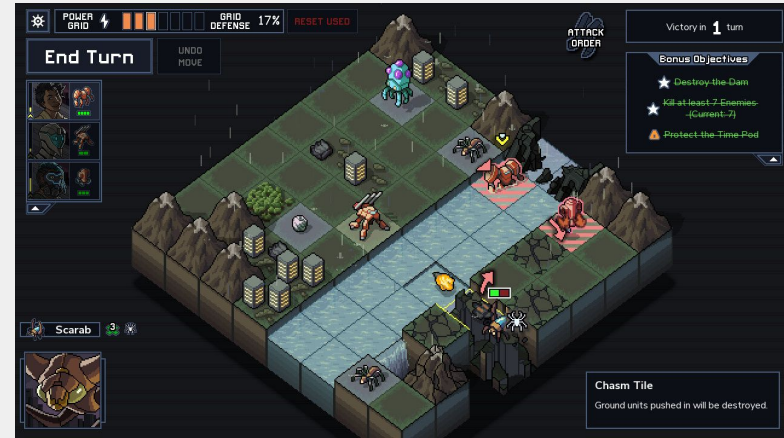
Demo 1

UG_HF2

IOWA STATE
UNIVERSITY

Into The Breach by Subset Games

- Turn-based strategy game on an 8x8 grid map.
- The objective is to protect structures that keep your team alive.
- All enemy attacks can be seen preemptively. You can also see when and where new enemies will spawn.
- Player controls up to three mechs to fight enemies.
- Each entity (PCs and NPCs) have a position, HP, a move range, status effects, and weapons.
- [Copyrighted Terms of Use](#)





Into The Breach by Subset Games

- The game state consists of a turn index, a power grid value, mission objectives, and an 8x8 map of the environment. [1]
- Each player character has a state consisting of a position, HP, move range, traits (flying, burrowing, etc.) and available actions (movement/weapons). [1]
- Each enemy has a state similar to the player characters but with an action intent. [1]
- Rewards include buildings and mechs being damaged or destroyed (+), enemies being damaged or destroyed (-), objectives being completed (-), etc. [1]

Chosen Game: Into The Breach by Subset Games

- We chose to create a prototype similar to Into The Breach.
- We have our four roles planned out: Game Loop & API, Entities, Map Design, and UI/Save & Load/Data
- We expect a prototype to be done within 2 weeks with a functional API for the other team to use.
- The prototype will be in Python because it allows for the modeling team to take our project as a package and make API calls without any slow HTTP requests.
- Name: TBD



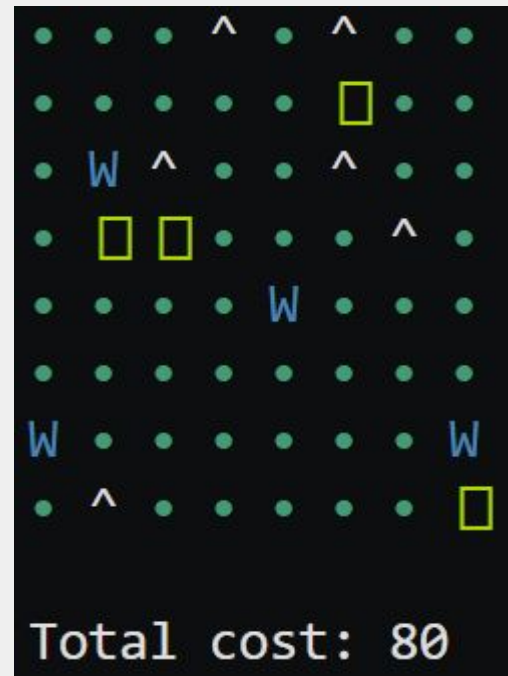
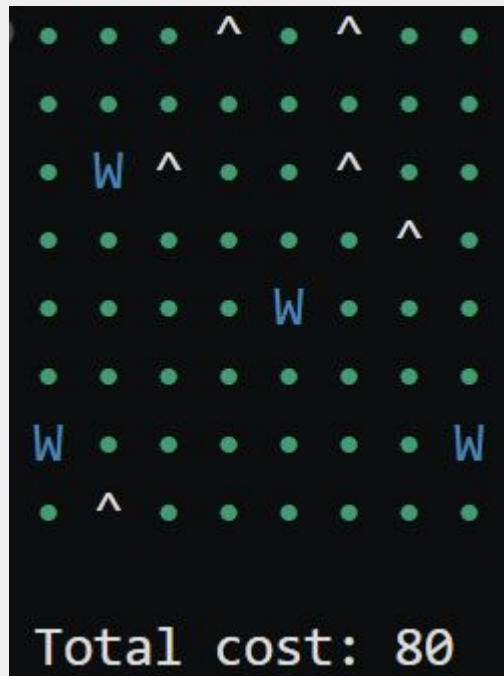
Map

- Map is procedurally generated every round
- Follows different rules depending on progress
- Main Tiles
 - Buildings - These are the tiles you work to protect
 - Ground - Normal tiles that have no special properties
 - Mountain - Impassable tiles that deal high damage
 - Water - Tiles that instantly eliminate certain enemies



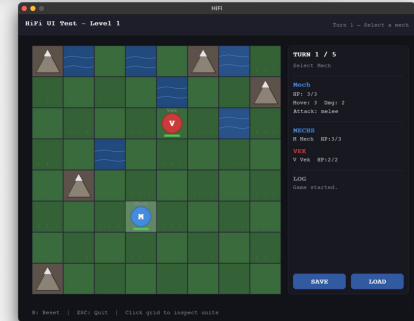
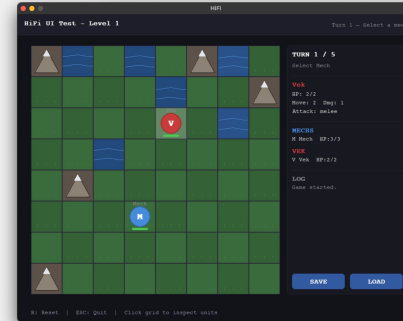
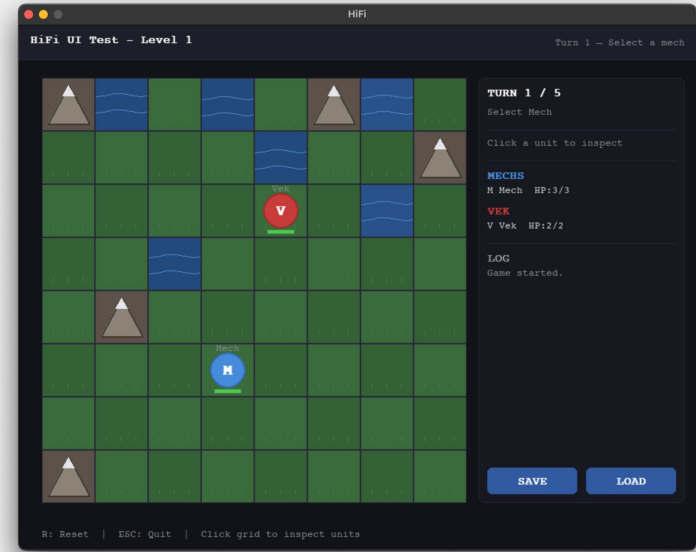
Map

- Simple Python Prototype
 - • - Normal Ground Tiles
 - W - Water Tiles
 - ^ - Mountain Tiles
 - □ - Building Tiles
- Cost
 - Ground and Building cost 1
 - Water cost 2
 - Mountain cost 3
 - Total of 80 ensures a weighted, randomly generated map that isn't too crowded



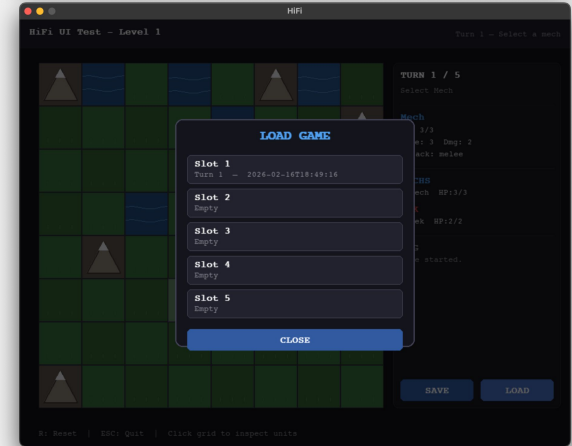
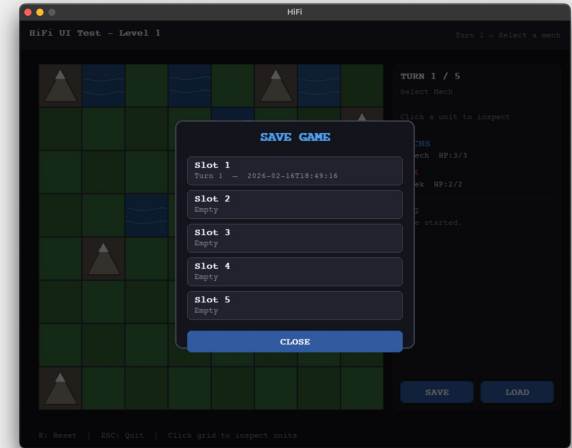
UI, Saving & Loading

- Uses mapcode.py and pygame to draw visual UI of the generated map
- adapter.py translates mapcode.py class objects into integers for saving and loading
- ui.py uses pygame to render/draw each tile with visual effects



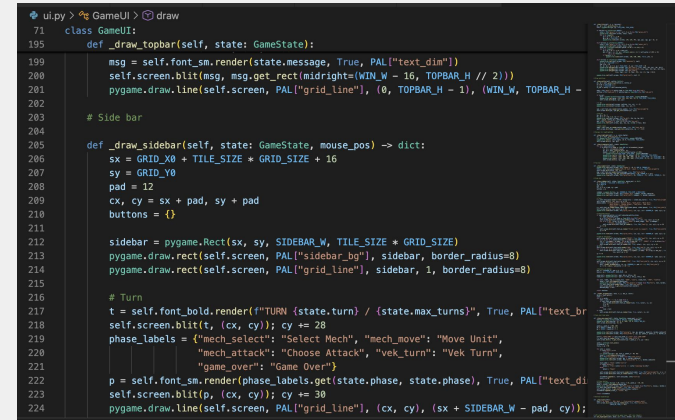
UI, Saving & Loading

- saveload.py handles saving and loading using JSON. Saves map, entities, turn info, etc.
- data.py defines the structures of the data used for saving and loading



UI, Saving & Loading - Challenges

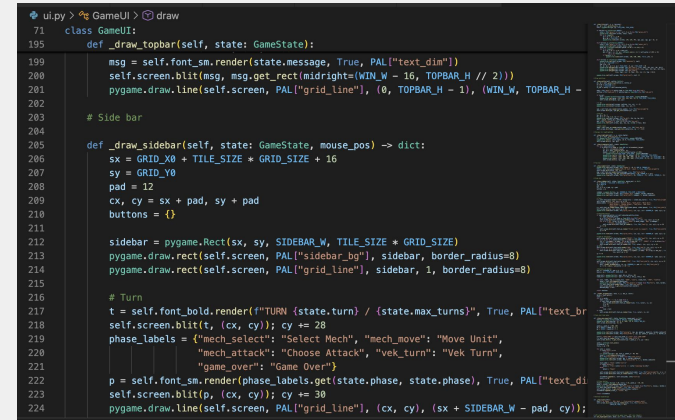
- Keeping code concise, readable and organized while building off of team members
- Implementing more code as time goes on
 - Scalability in different game features
- Working with teammates to merge more future entity code, game loop code, map code
 - Specifically game loop and API
- Future GoDot implementation



```
ul.py > GameUI > draw
71 class GameUI:
195     def _draw_topbar(self, state: GameState):
199         msg = self.font_sm.render(state.message, True, PAL["text_din"])
200         self.screen.blit(msg, msg.get_rect(midright=(WIN_W - 16, TOPBAR_H // 2)))
201         pygame.draw.line(self.screen, PAL["grid_line"], (0, TOPBAR_H - 1), (WIN_W, TOPBAR_H - 1))
202
203     # Side bar
204
205     def _draw_sidebar(self, state: GameState, mouse_pos) -> dict:
206         sx = GRID_X0 + TILE_SIZE * GRID_SIZE + 16
207         sy = GRID_Y0
208         pad = 12
209         cx, cy = sx + pad, sy + pad
210         buttons = {}
211
212         sidebar = pygame.Rect(sx, sy, SIDEBAR_W, TILE_SIZE * GRID_SIZE)
213         pygame.draw.rect(self.screen, PAL["sidebar_bg"], sidebar, border_radius=8)
214         pygame.draw.rect(self.screen, PAL["grid_line"], sidebar, 1, border_radius=8)
215
216         # Turn
217         t = self.font_bold.render(f"TURN {state.turn} / {state.max_turns}", True, PAL["text_br"])
218         self.screen.blit(t, (cx, cy)); cy += 28
219         phase_labels = {"mech_select": "Select Mech", "mech_move": "Move Unit",
220                        "mech_attack": "Choose Attack", "vek_turn": "Vek Turn",
221                        "game_over": "Game Over"}
222         p = self.font_sm.render(phase_labels.get(state.phase, state.phase), True, PAL["text_d"])
223         self.screen.blit(p, (cx, cy)); cy += 30
224         pygame.draw.line(self.screen, PAL["grid_line"], (cx, cy), (cx + SIDEBAR_W - pad, cy));
```

UI, Saving & Loading - Future timeline

- Implement entities once coded
- Interleave game loop mechanics
- Add UI for more game features
 - Buildings, objectives, stat bars, etc
- Use GoDot for 3D?
- More elegant graphics



```
ui.py > GameUI > draw
71 class GameUI:
195     def _draw_topbar(self, state: GameState):
199         msg = self.font_sm.render(state.message, True, PAL["text_din"])
200         self.screen.blit(msg, msg.get_rect(midright=(WIN_W - 16, TOPBAR_H // 2)))
201         pygame.draw.line(self.screen, PAL["grid_line"], (0, TOPBAR_H - 1), (WIN_W, TOPBAR_H - 1))
202
203     # Side bar
204
205     def _draw_sidebar(self, state: GameState, mouse_pos) -> dict:
206         sx = GRID_X0 + TILE_SIZE * GRID_SIZE + 16
207         sy = GRID_Y0
208         pad = 12
209         cx, cy = sx + pad, sy + pad
210         buttons = {}
211
212         sidebar = pygame.Rect(sx, sy, SIDEBAR_W, TILE_SIZE * GRID_SIZE)
213         pygame.draw.rect(self.screen, PAL["sidebar_bg"], sidebar, border_radius=8)
214         pygame.draw.rect(self.screen, PAL["grid_line"], sidebar, 1, border_radius=8)
215
216         # Turn
217         t = self.font_bold.render(f"TURN {state.turn} / {state.max_turns}", True, PAL["text_br"])
218         self.screen.blit(t, (cx, cy)); cy += 28
219         phase_labels = {"mech_select": "Select Mech", "mech_move": "Move Unit",
220                        "mech_attack": "Choose Attack", "vek_turn": "Vek Turn",
221                        "game_over": "Game Over"}
222         p = self.font_sm.render(phase_labels.get(state.phase, state.phase), True, PAL["text_din"])
223         self.screen.blit(p, (cx, cy)); cy += 30
224         pygame.draw.line(self.screen, PAL["grid_line"], (cx, cy), (cx + SIDEBAR_W - pad, cy));
```